

CSA17 - Analytical Problem-Solving
Assessment Tool 2 - Constraint-Based Problem Solving | Solution

Name: Roshitha

Reg No: 192424400

Q1. Hospital Doctor Shift Scheduling

Problem Explanation

Three doctors (D1, D2, D3) must each be assigned exactly one of three shifts (Morning, Afternoon, Night), one doctor per shift, subject to: D1 cannot take the Night shift; D2's shift must occur earlier in the day than D3's shift; D3 cannot take the Morning shift; every doctor gets exactly one shift.

CSP Formulation

- Variables: D1, D2, D3
- Initial domains: D1 = {Morning, Afternoon}; D2 = {Morning, Afternoon, Night}; D3 = {Afternoon, Night} — the Night/Morning restrictions (constraints i and iv) are removed from the domains up front
- Constraint ii: $\text{order}(D2) < \text{order}(D3)$, where Morning < Afternoon < Night
- Constraint iii: $\text{all-different}(D1, D2, D3)$ — one doctor per shift
- Constraint v: every doctor is assigned exactly one shift (total assignment)

Approach: Backtracking with Forward Checking and MRV

The solver is backtracking search strengthened with two classic CSP techniques:

- Minimum-Remaining-Values (MRV) variable ordering: `select_unassigned_variable()` always picks the doctor whose domain currently has the fewest legal shifts left, so the most constrained doctor is decided first.
- Forward checking: `forward_check()` runs immediately after each tentative assignment. It removes the just-used shift from every other doctor's domain, and additionally prunes D3's domain (to shifts later than D2's) or D2's domain (to shifts earlier than D3's) as soon as either variable is bound, directly enforcing constraint ii ahead of time.

Because forward checking removes doomed values from future domains the moment they become invalid, illegal branches are pruned before they are ever tried — the search only needs to backtrack when forward checking leaves some doctor with an empty domain.

Step-by-Step Trace (program output)

```
Trying D1 = Morning
Trying D2 = Afternoon
Trying D3 = Night
```

```
Final Valid Schedule:
{'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}
```

Walkthrough

- MRV picks D1 first (domain size 2, tied with D3, first in variable order): tries Morning — forward checking removes Morning from D2 and D3's domains; neither becomes empty, so the branch is kept.
- MRV now picks D2 (domain size 2: Afternoon, Night): tries Afternoon — forward checking removes Afternoon from D3's domain and applies the ordering constraint (D3's shift must be later than Afternoon), leaving $D3 = \{\text{Night}\}$.
- Only D3 remains, forced to Night — domain size 1, no other option to try.
- All three doctors are now assigned with no constraint violated, so the assignment is returned immediately as the solution.

Final Valid Schedule

D1 = Morning, D2 = Afternoon, D3 = Night

No rejections or backtracks appear in the trace, because MRV combined with forward checking steered the search directly to a valid, fully consistent assignment on the very first attempt at each variable — this is the intended benefit of pairing these two techniques rather than using plain chronological backtracking.

Q2. Robot Grid Path Planning (5×5 Grid) — Breadth-First Search

Problem Explanation

A robot must move from Start $S = (0,0)$ to Goal $G = (4,4)$ on a 5×5 grid, moving Up, Down, Left or Right at a cost of 1 per move, without crossing obstacle cells.

Grid Layout

Navigation grid (0 = free, 1 = obstacle), rows top to bottom:

Row0: 0 0 0 1 0

Row1: 0 1 0 1 0

Row2: 0 1 0 0 0

Row3: 0 1 1 1 0

Row4: 0 0 0 0 0

Approach: Breadth-First Search

`bfs_with_frontier_log()` implements genuine, uninformed BFS: the frontier is a `collections.deque` used strictly First-In-First-Out (`popleft()` to expand, `append()` to add), so nodes are explored purely in the order they were discovered — no cost or heuristic value influences that order. The Manhattan distance $h(n)$ is computed and printed alongside every expansion purely as a diagnostic/analysis value; it plays no role in choosing which node is expanded next. Because every move costs exactly 1, plain BFS is already guaranteed to return the shortest (least-cost) path, so an uninformed strategy is appropriate here.

Step-by-Step Node Expansion and Frontier Updates

Expand step 1: node=(0,0), $h(n)=8$, frontier_before=[]

Expand step 2: node=(0,1), $h(n)=7$, frontier_before=[(1,0)]

Expand step 3: node=(1,0), $h(n)=7$, frontier_before=[(0,2)]

Expand step 4: node=(0,2), h(n)=6, frontier_before=[(2,0)]
 Expand step 5: node=(2,0), h(n)=6, frontier_before=[(1,2)]
 Expand step 6: node=(1,2), h(n)=5, frontier_before=[(3,0)]
 Expand step 7: node=(3,0), h(n)=5, frontier_before=[(2,2)]
 Expand step 8: node=(2,2), h(n)=4, frontier_before=[(4,0)]
 Expand step 9: node=(4,0), h(n)=4, frontier_before=[(2,3)]
 Expand step 10: node=(2,3), h(n)=3, frontier_before=[(4,1)]
 Expand step 11: node=(4,1), h(n)=3, frontier_before=[(2,4)]
 Expand step 12: node=(2,4), h(n)=2, frontier_before=[(4,2)]
 Expand step 13: node=(4,2), h(n)=2, frontier_before=[(3,4),(1,4)]
 Expand step 14: node=(3,4), h(n)=1, frontier_before=[(1,4),(4,3)]
 Expand step 15: node=(1,4), h(n)=3, frontier_before=[(4,3),(4,4)]
 Expand step 16: node=(4,3), h(n)=1, frontier_before=[(4,4),(0,4)]
 Expand step 17: node=(4,4), h(n)=0, frontier_before=[(0,4)]

Optimal Path: [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
 Cost: 8

Result

Optimal path: (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4).
 Total cost = 8, matching the Manhattan-distance lower bound $h(0,0) = 8$ shown in the trace, which confirms the path found is optimal.

Q3. Autonomous Rescue Robot in a Partially Observable Environment

(a) Effect of Constraints on Decision-Making

- Partial observability (sense_adjacent() only reveals the current cell's immediate neighbours) forces the robot to plan with incomplete information and revise its plan as new cells are discovered, rather than computing one fixed route in advance.
- Blocked cells only remove edges from the search graph once they are sensed; known_map starts with just the start cell and grows one sensing step at a time.
- Risky zones (extra cost of 2, so a total step cost of 3) bias the agent toward paths that avoid them whenever an equally short safe alternative exists, since the online UCS re-plan always minimizes cumulative known cost.
- The online-search requirement means the robot cannot pre-compute a full offline plan; sense → plan → act → sense again is repeated every single step (online_ucs_rescue()'s main loop).

(b) Solution Approach

An Online Search Agent built around repeated Uniform Cost Search re-planning is used. At every position the robot senses its current cell's neighbours (sense_adjacent()), merges any newly observed cell types into known_map, then re-runs UCS from its current position toward the goal using only what is currently known (any cell not yet sensed defaults to free/cost-1 via known_map.get(neighbour, 0)). It commits to only the first step of the freshly computed route, moves there, and repeats. Because UCS expands the lowest-cost node first, and risky cells cost 3 rather than 1, the planner naturally prefers non-risky routes whenever one of equal or shorter cost exists.

(c) AI Concepts Demonstrated

- Intelligent agent: the robot perceives (`sense_adjacent()`), reasons (re-plans with UCS over `known_map`) and acts (moves to `route[1]`), forming a complete sense-plan-act loop each iteration.
- Rationality: at every cycle the agent chooses the action expected to minimize total path cost given everything it currently knows — it behaves rationally with respect to its available information, even though that information is incomplete.
- Environment type: partially observable (limited sensing radius), effectively dynamic-by-discovery from the agent's point of view (the known map changes as new areas are sensed, even though the underlying grid itself is static), sequential (each move affects future options), discrete (grid cells and discrete actions), and single-agent.

(d) Justification

Repeated UCS re-planning is well suited here because UCS is guaranteed optimal for non-negative edge costs, which matches the normal-cost/risky-cost structure of this problem, and re-invoking it after every sensing step is the standard way to turn an offline-optimal algorithm into an online agent that reacts to newly discovered obstacles and risk without needing full prior knowledge of the map. A single offline UCS run would be inapplicable since the map is unknown in advance, and a purely reactive agent with no re-planning could easily walk into dead ends or unnecessarily expensive risky cells.

Environment Layout

True environment (0 = free, 1 = obstacle, 2 = risky zone), rows top to bottom:

Row0: 0 2 0 0 0

Row1: 0 1 0 1 2

Row2: 0 0 0 1 0

Row3: 2 1 0 0 0

Row4: 0 0 2 0 0

Trace and Result (program output)

At (0,0), sensed neighbours: {(0,1): 2, (1,0): 0}

At (1,0), sensed neighbours: {(1,1): 1, (2,0): 0, (0,0): 0}

At (2,0), sensed neighbours: {(2,1): 0, (3,0): 2, (1,0): 0}

At (2,1), sensed neighbours: {(2,2): 0, (3,1): 1, (2,0): 0, (1,1): 1}

At (2,2), sensed neighbours: {(2,3): 1, (3,2): 0, (2,1): 0, (1,2): 0}

At (3,2), sensed neighbours: {(3,3): 0, (4,2): 2, (3,1): 1, (2,2): 0}

At (3,3), sensed neighbours: {(3,4): 0, (4,3): 0, (3,2): 0, (2,3): 1}

At (3,4), sensed neighbours: {(4,4): 0, (3,3): 0, (2,4): 0}

Least-Cost Path: [(0,0), (1,0), (2,0), (2,1), (2,2), (3,2), (3,3), (3,4), (4,4)]

Total Cost: 8

Path travelled: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4).

Total cost incurred: 8. Every cell on this route is a normal (cost-1) cell — although risky cells such as (0,1), (3,0), (4,2) and (1,4) exist nearby and were sensed along the way, the online UCS re-planning always found a same-cost normal route around them, so no risky-zone

penalty was ever incurred. This shows the sense-plan-act loop successfully steering the agent away from added risk purely through cost-minimizing re-planning, without needing to know the whole map in advance.